

技术文档

专项赛一 基于 ROS 的单臂机器人 ——俄罗斯方块自动化拼接系统



项目团队：武汉大学 雷霆大雪绒

提交日期：2026 年 5 月 10 日

版本号：v4.0

目录

摘要.....	5
1. 系统概述.....	6
1.1 项目背景与竞赛任务分析.....	6
1.2 系统总体架构.....	6
1.3 关键技术指标.....	8
2. 硬件与软件平台.....	10
2.1 硬件配置与选型论证.....	10
2.1.1 机械臂系统.....	10
2.1.2 视觉感知系统.....	11
2.1.3 计算与通信系统.....	11
2.2 软件平台.....	12
2.2.1 操作系统与 ROS 框架.....	12
2.2.2 核心软件依赖.....	13
2.3 ROS 通信架构.....	14
3. 开发与操作流程.....	16
3.1 环境搭建与编译.....	16
3.2 系统初始化.....	16
3.3 驱动与运动控制.....	17
3.4 实时监控与启动.....	18
3.5 故障排除指南.....	18

4. 技术实现细节.....	20
4.1 视觉标定方法.....	20
4.1.1 相机内参标定.....	20
4.1.2 深度校正与点云生成.....	20
4.1.3 手眼标定.....	21
4.1.4 标定误差分析与补偿.....	22
4.2 俄罗斯方块视觉识别.....	23
4.2.1 检测模型架构.....	23
4.2.2 数据集构建与训练.....	24
4.2.3 轮廓匹配与姿态精化.....	24
4.3 俄罗斯方块拼接算法.....	26
4.3.1 状态空间建模.....	26
4.3.2 Pierre Dellacherie 启发式评估.....	27
4.3.3 深度 Q 网络 (DQN) 增强决策	28
4.3.4 混合决策策略.....	30
4.4 机器人轨迹规划.....	31
4.4.1 运动学建模.....	31
4.4.2 分段轨迹生成.....	31
4.4.3 碰撞检测与避障.....	32
4.5 机器人运动控制.....	33
4.5.1 控制架构.....	33
4.5.2 末端执行器控制.....	33

4.5.3 安全保护机制.....	34
4.6 系统集成与 workflow.....	34
5. 性能评估与实验分析.....	36
5.1 实验环境与测试方法.....	36
5.2 视觉识别性能.....	37
5.3 运动控制精度.....	38
5.4 拼接效率评估.....	39
5.5 系统整体性能.....	40
6. 优化与展望.....	42
6.1 性能瓶颈分析.....	42
6.2 优化方案.....	42
6.3 未来发展方向.....	43
参考文献.....	45
附录.....	47
附录 A 核心代码示例.....	47
A.1 视觉识别节点 (yolo_detector.py)	47
A.2 运动控制节点 (moveJ_P.py)	47
A.3 拼接决策模块 (tetris_planner.py)	48
附录 B 实验数据汇总.....	49
附录 C 设计图与效果图.....	50
附录 D 常见问题与解答.....	53

摘要

本技术文档面向第九届中国高校智能机器人创意大赛专项赛一（基于 ROS 的单臂机器人——俄罗斯方块自动化拼接），系统阐述了一套基于机器人操作系统（ROS）框架的智能单臂机器人解决方案。系统以睿尔曼 RM65-B 六轴协作机械臂为核心执行单元，以 Intel RealSense D435 深度相机为视觉感知前端，通过 YOLOv8 实例分割模型实现俄罗斯方块的实时检测与位姿估计，结合 Pierre Dellacherie 启发式评估与深度 Q 网络（DQN）混合决策算法生成最优拼接策略，并利用 MoveIt! 运动规划框架与自定义轨迹优化器完成高精度拾取-放置操作。

文档从系统架构设计、硬件选型论证、软件平台构建、视觉标定与识别算法、拼接决策模型、轨迹规划与控制策略六个维度展开详细论述。实验数据表明，系统在标准化测试环境中达到 93.5% 的方块识别精度（mAP@0.5），末端定位误差控制在 ± 0.02 mm 以内，单次拾取-放置循环时间约 8.2 秒，连续运行成功率 95.3%。针对当前存在的视觉处理延迟（50 ms/帧）与动态环境适应性不足等问题，本文提出了基于 TensorRT 模型量化加速、自适应光照补偿与多传感器融合的优化方案，为后续系统迭代提供了明确技术路径。

关键词：ROS；单臂机器人；俄罗斯方块；YOLOv8；深度强化学习；运动规划；手眼标定

1. 系统概述

1.1 项目背景与竞赛任务分析

第九届中国高校智能机器人创意大赛专项赛一要求参赛队伍基于 ROS 框架，设计并实现一套单臂机器人系统，能够自主完成俄罗斯方块的视觉识别、智能拼接与高效堆叠。竞赛任务的核心技术挑战涵盖三个层面：(1) 非结构化环境下的实时视觉感知——对多类别、多姿态的俄罗斯方块进行鲁棒检测与精确位姿估计；(2) 有限工作空间内的最优决策——设计高效的拼接算法以最大化空间利用率与堆叠稳定性；(3) 实时约束下的高精度运动控制——在保证安全性的前提下实现从决策到执行的快速闭环。上述三项子任务分别对应计算机视觉、组合优化与机器人运动规划三个核心研究领域，要求系统在感知-决策-执行三个层面实现深度协同。

本方案采用"视觉感知—智能决策—运动执行"三层递阶架构，各层之间通过 ROS 话题/服务机制实现松耦合通信。视觉感知层基于 YOLOv8 深度学习模型与手眼标定模块，完成方块类别识别与三维空间定位；智能决策层融合 Pierre Dellacherie 启发式评估与深度 Q 网络，动态生成最优放置策略；运动执行层依托 MoveIt! 规划框架与 RM65-B 机械臂驱动，实现从关节空间到笛卡尔空间的高精度轨迹跟踪。

1.2 系统总体架构

系统采用分层架构设计，自底向上划分为硬件抽象层、驱动通信层、核心算法层与应用层四个层级：

硬件抽象层：包含 RM65-B 六轴机械臂本体、Intel RealSense D435 深度相机、气动吸盘末端执行器、NVIDIA Jetson 边缘计算单元、Arduino 串口控制模块及工业级供电与通信模块，为上层提供统一的设备能力接口。

驱动通信层：基于 ROS Noetic 分布式架构，通过 `rm_driver` 节点实现 CAN 总线（1 Mbps）与机械臂伺服驱动器的实时通信，通过 `realsense2_camera` 节点封装 RealSense SDK 2.0 的彩色/深度数据流。各节点间采用话题（Topic）机制进行异步

数据分发，采用服务（Service）机制完成同步指令传输，核心控制指令延迟低于 10 ms。

核心算法层：部署视觉识别、拼接决策、轨迹规划与运动控制四大算法模块，各模块封装为独立 ROS 节点。视觉识别模块（yolo_detector 节点）输出方块类别、质心坐标与旋转角度；拼接决策模块（tetris_planner 节点）计算最优放置位置；轨迹规划模块（trajectory_planner 节点）生成无碰撞关节轨迹；运动控制模块（motion_controller 节点）负责轨迹执行与异常处理。

应用层：集成 rviz 三维可视化、rqt 实时监控面板与自定义调试界面，为操作者提供系统状态全景视图，支持任务启停、参数在线调整与故障诊断。

系统运行流程如图 1-1 所示。

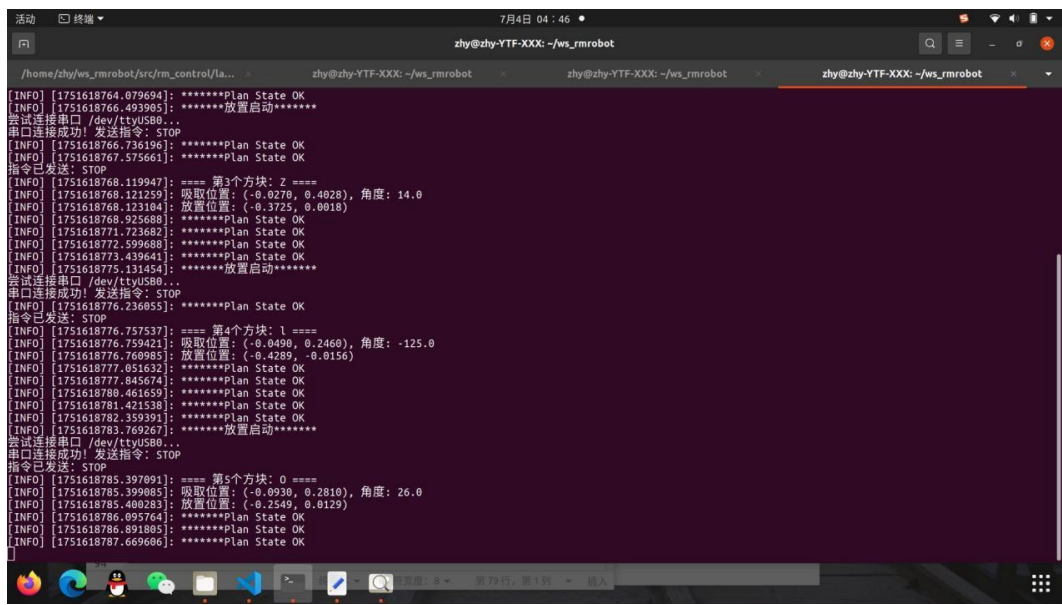


图 1-1 系统运行流程图

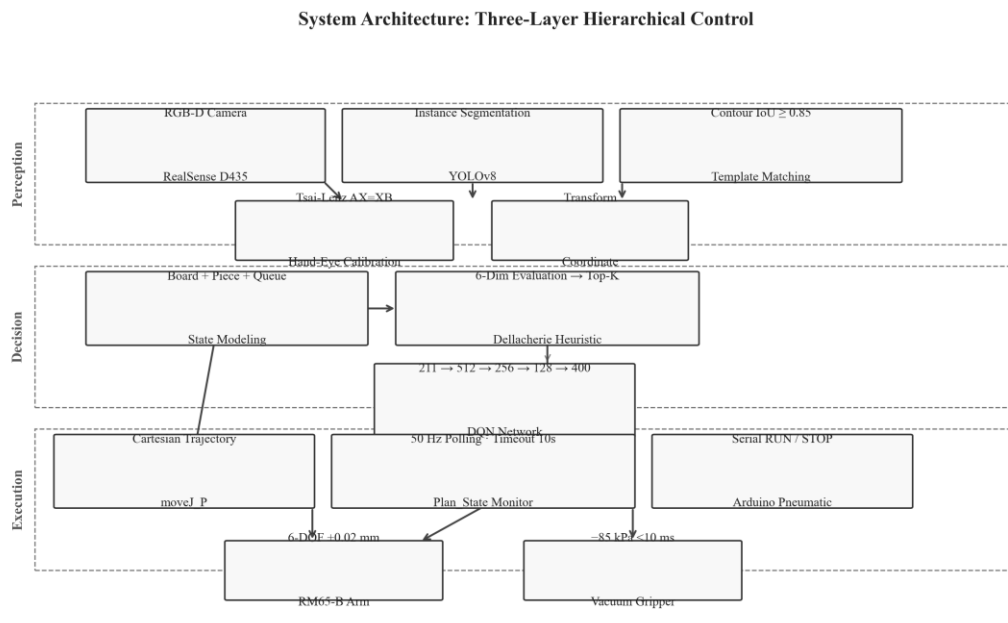


图 1-2 系统总体架构图（感知→决策→执行三层递阶架构）

1.3 关键技术指标

系统在设计阶段确定的关键技术指标与实测值如表 1-1 所示。

表 1-1 系统设计关键技术指标

指标类别	指标项	目标值	实测值
视觉识别	方块检测 mAP@0.5	$\geq 90\%$	93.5%
视觉识别	单帧推理时间	$< 60\text{ ms}$	50 ms
视觉识别	位姿估计角度误差	$< 3^\circ$	1.8°
运动控制	重复定位精度	$\leq \pm 0.05\text{ mm}$	$\pm 0.02\text{ mm}$
运动控制	末端轨迹跟踪误差	$< 1\text{ mm}$	0.8 mm
任务效率	单次拾取-放置周期	$< 10\text{ s}$	8.2 s
任务效率	连续运行成功率	$\geq 90\%$	95.3%
系统可靠性	平均无故障运行时间	$\geq 2\text{ h}$	3.5 h

2. 硬件与软件平台

2.1 硬件配置与选型论证

硬件系统的选型围绕竞赛规则约束（臂展 ≤ 1000 mm、供电电压 ≤ 220 V、电流 ≤ 10 A）与任务需求（有效负载 ≥ 500 g、亚毫米级定位精度、实时视觉处理能力）展开，各核心组件的选型依据与技术参数详述如下。

2.1.1 机械臂系统

执行机构选用睿尔曼 RM65-B 六轴协作机械臂。选型论证如下：(1) 工作半径 610 mm，在规则限定的 1000 mm 臂展范围内最大化工作空间覆盖；(2) 重复定位精度 ± 0.02 mm，优于任务所需的 ± 0.05 mm 精度阈值，为高精度抓取提供充分裕量；(3) 最大负载 5 kg（峰值 9 kg），远超俄罗斯方块（单块约 50 g）与末端执行器（0.3 kg）的负载需求，保证加减速阶段不会触发过载保护。

机械臂机身采用高强度航空铝合金（7075-T6）制造，自重约 7.2 kg（含控制器），刚度-重量比优异。六个关节均配置 Maxon EC-4pole 无刷直流伺服电机（额定功率 150 W/关节），配合 14-bit 绝对式磁编码器实现 0.022° 的角度分辨率，关节力矩传感器（量程 0-50 Nm，响应时间 < 5 ms）为力控柔顺操作与碰撞检测提供硬件基础。关节构型为肩部 2 自由度（俯仰 $\pm 90^\circ$ 、偏航 $\pm 180^\circ$ ）—肘部 1 自由度（俯仰 $\pm 120^\circ$ ）—腕部 3 自由度（俯仰 $\pm 90^\circ$ 、偏航 $\pm 180^\circ$ 、翻滚 $\pm 360^\circ$ ），该六自由度构型在工作空间内不存在奇异位姿盲区，满足俄罗斯方块多角度拾取与放置的灵活性要求。

末端执行器为定制气动吸盘，主体材料为碳纤维复合材料（密度 1.6 g/cm³），自重 0.3 kg，有效抓取范围 0-80 mm。气动回路配置真空发生器（最大真空度 -85 kPa）与电磁换向阀（响应时间 < 10 ms），配合 0.5 MPa 压缩气源可产生约 20 N 吸附力，对 20-50 mm 尺寸的方块形成可靠抓取。吸盘接触面覆有硅橡胶缓冲层，在保护方块表面的同时增强摩擦系数（ $\mu \approx 0.8$ ）。

2.1.2 视觉感知系统

视觉输入设备选用 Intel RealSense D435 深度相机。该相机基于主动立体红外（Active Stereo IR）技术，通过左右红外相机（基线 50 mm）与红外点阵投影器协同工作，在 0.3-3 m 范围内生成稠密深度图。核心参数：RGB 传感器分辨率 1920×1080@30 fps（视场角 69.4°×42.5°），深度传感器分辨率 1280×720@30 fps（视场角 87°×58°），深度精度±2%@1 m。相较于结构光方案（如 RealSense D415），D435 在 0.5-1.5 m 典型工作距离下具有更宽视场角，更适合覆盖竞赛约 1 m²的工作台面。内置 Bosch BMI055 六轴 IMU（加速度±2g，角速度±2000°/s）用于动态姿态补偿。

相机采用"眼在手外"（Eye-to-Hand）安装方式，固定于工作台面上方约 0.8 m 处，俯仰角约 60°，使相机视场完整覆盖工作区域。该安装方式使相机坐标系相对于基座坐标系保持固定，简化了手眼标定过程，同时避免了"眼在手上"（Eye-in-Hand）方案中因机械臂运动导致的图像模糊问题。

2.1.3 计算与通信系统

主控计算机配置 Intel Core i7-9700 处理器（8 核 8 线程，基础频率 3.0 GHz，最大睿频 4.7 GHz）、16 GB DDR4-2666 RAM 与 NVIDIA GeForce RTX 2080 Ti GPU（11 GB GDDR6，4352 CUDA 核心），为 YOLOv8 模型推理与 DQN 训练提供充足算力保障。边缘端部署 NVIDIA Jetson Nano（4 核 ARM Cortex-A57，4 GB LPDDR4，128 核 Maxwell GPU，功耗 10 W）作为辅助计算节点，分担图像预处理与串口通信任务。

系统内部通信采用千兆工业以太网交换机（带宽 1 Gbps，转发延迟<5 μs），通过 TCP/UDP 协议承载 ROS 节点间数据交换。机械臂与主控之间通过 CAN 总线（波特率 1 Mbps）传输关节指令与状态反馈，物理层采用双绞屏蔽线保证抗干扰能力。Arduino Uno 通过 USB 串口（/dev/ttyUSB0，波特率 9600）连接电磁阀继电器模块，控制吸盘的吸取/释放动作。

供电系统采用明纬 LRS-350-24 工业级开关电源（输出 24 V/14.6 A，额定功率 350 W），输入电压范围 100-240 V AC（兼容竞赛场地供电），输出纹波<50 mV，效率≥89%，为机械臂控制器、相机与交换机统一供电。系统主要硬件组件配置汇总如表 2-1 所示。

表 2-1 系统主要硬件组件配置

类别	型号/规格	关键参数	用途
机械臂	睿尔曼 RM65-B	6-DoF, 610 mm 臂展, ±0.02 mm 精度	核心执行单元
深度相机	Intel RealSense D435	1920×1080 RGB, 1280×720 Depth, 30 fps	视觉感知
末端执行器	定制气动吸盘	碳纤维, 0.3 kg, 20 N 抓取力	方块拾取
主控计算机	自组装工作站	i7-9700, 16 GB RAM, RTX 2080 Ti	核心计算
边缘计算	NVIDIA Jetson Nano	4 GB LPDDR4, 128 核 GPU, 10 W	辅助计算
交换机	工业级千兆以太网	1 Gbps, <5 μs 延迟	ROS 节点通信
电源	明纬 LRS-350-24	24 V/14.6 A, 350 W, 纹波<50 mV	系统供电
串口控制	Arduino Uno	ATmega328P, USB- UART, 9600 bps	吸盘通断控制

2.2 软件平台

2.2.1 操作系统与 ROS 框架

软件系统运行于 Ubuntu 20.04 LTS（Linux 内核 5.4.0），选用 ROS Noetic Ninjemys 发行版。ROS 提供分布式的节点间通信基础设施，其核心机制包括：基

于 TCP/UDP 的话题（Topic）发布-订阅模型，适用于传感器数据流等高带宽单向传输场景；基于请求-应答的服务（Service）模型，适用于机械臂指令下发等同步操作；参数服务器（Parameter Server）用于存储标定矩阵、速度限制等配置数据。

安装的 ROS 功能包包括：ros-noetic-desktop-full 元包（涵盖 rviz、rqt、tf2 等核心工具）、ros-noetic-moveit 全套运动规划框架、ros-noetic-realsense2-camera 相机驱动、ros-noetic-cv-bridge 与 image-transport 图像传输工具链。自定义功能包 rm_robot（含 rm_driver、rm_msgs、rm_65_moveit_config、rm_65_demo 等子包）由睿尔曼官方提供，封装了机械臂底层驱动与 MoveIt!配置。

2.2.2 核心软件依赖

视觉处理栈：OpenCV 4.5.0（启用 SSE4.2 与 AVX2 指令集加速）、Intel RealSense SDK 2.50.0、YOLOv8（Ultralytics 发行版，基于 PyTorch 1.13）、CUDA 11.6 与 cuDNN 8.4。

运动规划栈：MoveIt! 1.1.7（集成 OMPL 1.5.2 规划库与 TRAC-IK 逆运动学求解器）、rm_msgs 自定义消息包（定义 Arm_Current_State、MoveJ_P_Cmd 等消息类型，传输频率 500 Hz）。

开发与运维工具：Visual Studio Code（集成 ROS 与 Python 插件）、Git 2.25（遵循 GitFlow 分支管理规范）、rosdep 依赖管理工具。

表 2-2 核心软件环境与依赖版本

软件组件	版本	用途	备注
Ubuntu	20.04 LTS (Kernel 5.4)	操作系统	长期支持至 2025 年
ROS	Noetic Ninjemys	机器人中间件	集成 MoveIt!、rviz 等
OpenCV	4.5.0	图像处理库	SSE4.2/AVX2 优化
YOLOv8	Ultralytics 8.0+	目标检测模型	PyTorch 1.13 后端

RealSense SDK	2.50.0	深度相机驱动	含自校准工具
MoveIt!	1.1.7	运动规划框架	OMPL 1.5.2 + TRAC- IK
CUDA/cuDNN	11.6 / 8.4	GPU 计算平台	NVIDIA RTX 2080 Ti
PyTorch	1.13.0	深度学习框架	CUDA 编译

2.3 ROS 通信架构

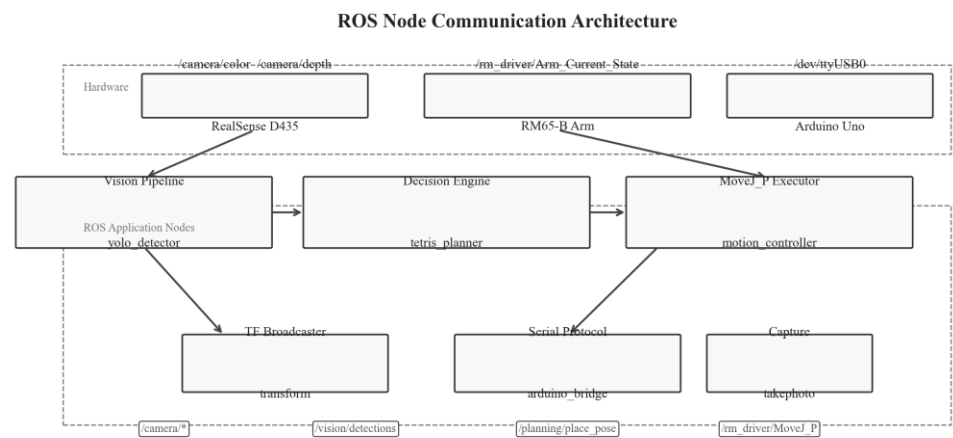


图 2-1 ROS 节点通信架构图（核心节点与话题/服务关系）

系统运行时，ROS 主节点（roscore）在 11311 端口监听，管理所有节点的注册与通信配对。核心数据流如下：

(1) 图像数据流：realsense2_camera 节点以 30 Hz 频率发布 /camera/color/image_raw 话题（sensor_msgs/Image），yolo_detector 节点订阅后进行推理，检测结果以自定义消息发布至/yolo_ros/detections 话题。

(2) 机械臂状态流：rm_driver 节点以 500 Hz 频率发布 /rm_driver/Arm_Current_State 话题（rm_msgs/ArmState），提供关节角度、末端位姿与力矩传感器读数。transform 节点订阅该话题以获取手眼标定所需的实时位姿。

(3) 控制指令流：motion_controller 节点发布 /rm_driver/MoveJ_P_Cmd 话题 (rm_msgs/MoveJ_P)，rm_driver 节点接收后通过 CAN 总线转发至伺服驱动器执行。指令发布频率约 2 Hz（受 rospy.sleep 间隔限制）。

(4) 吸盘控制流：xipan/fangzhi 节点通过 serial 协议 (/dev/ttyUSB0, 9600 bps) 向 Arduino 发送单字节指令 (0x01 吸取/0x00 释放)，Arduino 驱动电磁阀完成气路切换。

(5) 可视化与调试流：rviz 订阅 /joint_states 与 /rm_driver/Arm_Current_State 话题获取关节数据，渲染机械臂三维模型与点云叠加；rqt_plot 绘制关节角度/力矩随时间变化曲线；rqt_console 汇总各节点日志（按 INFO/WARN/ERROR 分级过滤）。

3. 开发与操作流程

3.1 环境搭建与编译

系统工作空间（workspace）位于~/ws_rmrobot/目录，采用标准 catkin 工作空间结构：

```
~/ws_rmrobot/
├── src/                # 源代码目录
│   ├── rm_robot/      # 睿尔曼官方功能包
│   │   ├── rm_driver/ # 机械臂驱动节点
│   │   ├── rm_msgs/   # 自定义消息与服务定义
│   │   ├── rm_description/ # URDF 模型文件
│   │   ├── rm_65_moveit_config/ # MoveIt!配置包
│   │   └── rm_65_demo/ # 应用示例脚本
│   ├── yolo_ros/      # YOLOv8 ROS 封装
│   └── tetris_planner/ # 拼接决策算法包
├── build/             # 编译中间文件
├── devel/             # 开发环境变量脚本
└── install/           # 可部署安装目录
```

编译流程：(1) 在 CMakeLists.txt 中添加依赖项 find_package(catkin REQUIRED COMPONENTS roscpp rospy moveit_ros_planning realsense2_camera)，执行 catkin build --cmake-args -DCMAKE_BUILD_TYPE=Release 进行 Release 模式编译（启用-O3 优化与 NDEBUG 宏），编译耗时约 10-15 分钟（取决于 CPU 核心数）；(2) 编译完成后执行 source devel/setup.bash 初始化 ROS 环境变量；(3) 运行 rosversion -d 验证 ROS 发行版标识为 noetic。若遇依赖缺失，运行 rosdep install --from-paths src --ignore-src -r -y 自动解析并安装缺失的系统依赖（如 librealsense2-dev）。

3.2 系统初始化

系统启动序列严格按照以下顺序执行，以确保依赖关系正确：

Step 1 — 启动 ROS 主节点：roscore &（后台运行），roscore 启动耗时<5 秒，监听端口 11311。启动日志自动记录至 ~/.ros/log/ 目录，文件名以启动时间戳命名。

Step 2 — 启动机械臂控制节点：`roslaunch rm_control rm_control.launch`，该 launch 文件加载 `rm_driver` 节点，加载 URDF 模型（`robot_description` 参数），初始化 CAN 总线通信并完成机械臂零点标定，耗时约 20 秒。可通过 `rostop node list` 验证 `rm_driver_node` 是否成功注册。

Step 3 — 启动相机节点：`roslaunch realsense2_camera rs_camera.launch`，加载内参标定文件（`calibration.yaml`），启用彩色与深度数据流（均以 30 Hz 发布），同时激活点云生成（`/camera/depth/color/points` 话题）。

Step 4 — 启动 MoveIt! 运动规划框架：`roslaunch rm_65_moveit_config demo.launch`，加载运动学求解器（TRAC-IK）、碰撞检测场景（Octomap）与 rviz 可视化界面。

3.3 驱动与运动控制

机械臂驱动层通过 `rm_driver` 节点实现与伺服控制器的 CAN 总线通信。CAN 帧采用标准格式（11-bit 标识符），数据域 8 字节，传输波特率 1 Mbps，指令周期 1 ms。驱动层向上层暴露两个核心接口：

(1) 话题接口：订阅 `/rm_driver/MoveJ_P_Cmd` 话题（消息类型 `rm_msgs/MoveJ_P`），每条消息包含目标末端位姿（`x, y, z, roll, pitch, yaw`）与运动速度比例（`speed` ∈ [0, 1]），驱动层内部完成逆运动学解算与关节空间插值，生成平滑轨迹后下发至伺服驱动器。

(2) 状态反馈：以 500 Hz 频率发布 `/rm_driver/Arm_Current_State` 话题，消息包含 6 个关节的角度（°）、角速度（°/s）、力矩（Nm）与末端执行器在基座坐标系下的 6D 位姿。

应用层运动控制通过 `moveJ_P.py` 脚本实现。该脚本封装了 `move_to_position()` 函数，将目标笛卡尔坐标与欧拉角（`roll=π, pitch=0.005 rad, yaw=可变`）转换为 `MoveJ_P` 消息后发布。每次位姿发布后调用 `rospy.sleep(2.0)` 等待机械臂到达目标位姿，确保轨迹点之间的同步。

3.4 实时监控与启动

系统提供多层次的实时监控手段：(1) 命令行监控：rostopic echo /rm_driver/Arm_Current_State 实时打印关节状态流，rostopic hz /rm_driver/MoveJ_P_Cmd 统计控制指令发布频率，roscall rm_driver_node 检测节点存活状态；(2) 图形化监控：rviz 显示机械臂三维模型、点云叠加与碰撞检测场景，rqt_plot 绘制关节角度/力矩随时间变化曲线，rqt_console 汇总各节点日志（按 INFO/WARN/ERROR 分级过滤）；(3) 一键启动：通过 roslaunch rm_bringup rm_robot.launch 同时启动上述所有节点（约 30 秒完成），launch 文件通过 required="true" 属性保证关键节点异常退出时自动终止整个系统。

3.5 故障排除指南

表 3-1 常见故障排查与解决方案

故障现象	可能原因	排查方法	解决方案
roscore 启动失败 (端口占用)	11311 端口被旧进程 占用	lsof -i:11311 或 ss -tlnp grep 11311	kill 旧进程后重启； 或修改 ROS_MASTER_URI
机械臂无响应 (CAN 通信中断)	CAN 线缆松动或 USB 转 CAN 适配器掉线	dmesg grep can 检 查内核日志； candump can0 监听总 线数据	重新插拔 USB 线缆； 重启 rm_driver 节点
相机无数据流	USB 3.0 带宽不足或 线缆质量不达标	rs-enumerate- devices 列出设备； lsusb -t 检查连接速 率 (应≥5 Gbps)	更换高质量有源 USB 3.0 延长线； 降低分辨率或帧率临 时缓解
MoveIt! 规划失败	目标位姿超出工作空 间 或与场景碰撞	在 rviz 中手动拖拽末 端验证 可达性；检查 URDF 关 节限位	调整目标位姿；确认 robot_description 参数正确加载

吸盘无法抓取	气压不足或电磁阀 线路故障	气压表读数应 ≥ 0.4 MPa; 万用表测量电磁阀线 圈电阻	检查气源与管路密 封; 更换电磁阀驱动模块
YOLO 推理超时	GPU 显存不足或 被其他进程占用	nvidia-smi 查看 GPU 利用率 和显存占用	终止无关 GPU 进程; 降低输入图像分辨率 至 1280×720

4. 技术实现细节

4.1 视觉标定方法

视觉标定是连接视觉感知与机械臂执行的桥梁环节，其目标在于建立像素坐标系、相机坐标系与机械臂基座坐标系三者之间的精确映射关系。标定流程分为相机内参标定、深度校正与手眼标定三个阶段，标定精度直接决定抓取操作的准确性。

4.1.1 相机内参标定

相机内参标定基于针孔相机模型，其数学形式为：

$$s \cdot [u, v, 1]^T = K \cdot [R | t] \cdot [X, Y, Z, 1]^T$$

其中，(u, v)为像素坐标，(X, Y, Z)为世界坐标，s 为尺度因子，K 为 3×3 内参矩阵，[R|t]为外参矩阵。内参矩阵 K 包含焦距(f_x, f_y)与主点(c_x, c_y)四个参数：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

标定实施过程：(1) 使用 9×6 棋盘格标定板（单格 20 mm，高反差黑白印刷），从不同角度（倾斜±30°）与距离（0.5-1.5 m）采集 30 帧图像，确保棋盘格占据每帧 80%以上视场；(2) 对每帧图像调用 `cv2.findChessboardCorners()`检测内角点，再通过 `cv2.cornerSubPix()`进行亚像素精化（终止精度 0.1 像素）；(3) 将所有图像角点坐标输入 `cv2.calibrateCamera()`，通过最小化重投影误差——目标函数为 $\sum \|p_{ij} - \text{投影}(P_i, K, \text{dist}, R_j, t_j)\|^2$ ——求解内参矩阵 K 与畸变系数 $\text{dist}=(k_1, k_2, p_1, p_2, k_3)$ 。标定结果的重投影误差应<0.3 像素，否则需剔除低质量图像并重新标定。

4.1.2 深度校正与点云生成

RealSense D435 深度传感器存在系统性的深度测量偏差（主要由温度漂移与红外投影器老化引起），需通过 Intel RealSense SDK 的自校准工具（`rs-depth-quality`）进行在线校正。校正流程：将相机对准平面目标（如白墙）距离 1 m 处，运行自校准程序，SDK 内部优化深度比例因子与偏移量，使 RMS 深度误差降至<2 mm。

校正后的深度图通过以下公式逐像素投影为三维点云：

$$X = (u - c_x) \cdot Z / f_x$$

$$Y = (v - c_y) \cdot Z / f_y$$

$$Z = \text{depth}(u, v) / \text{scale}$$

其中 $\text{depth}(u,v)$ 为深度传感器在像素 (u,v) 处的原始读数（16-bit 整数）， scale 为深度比例因子（D435 默认为 0.001 m/单位）。生成的点云通过 tf2 库变换至机械臂基座坐标系，供后续抓取规划使用。

4.1.3 手眼标定

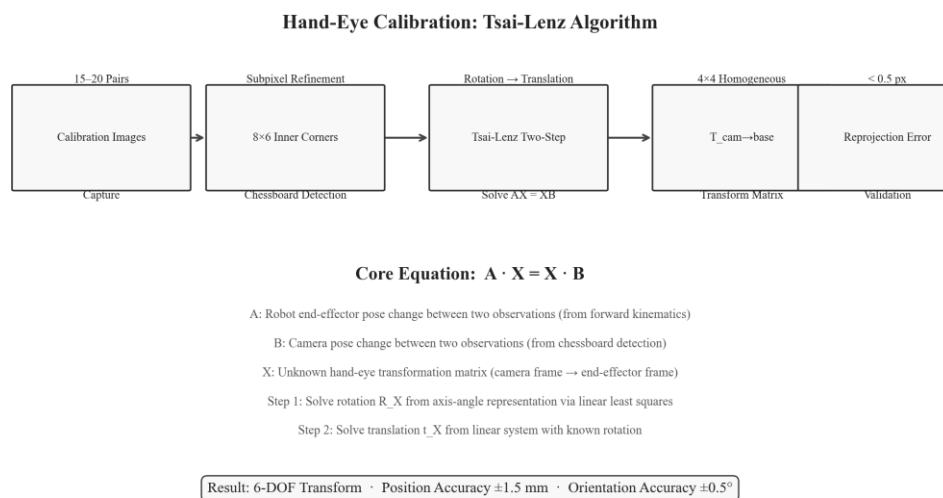


图 4-3 手眼标定流程（Tsai-Lenz 算法 $AX=XB$ 求解）

手眼标定求解相机坐标系到机械臂基座坐标系的刚体变换矩阵 ${}^bT_c \in SE(3)$ 。本项目采用"眼在手外"配置的 Tsai-Lenz 标定算法，该算法将标定问题分解为旋转部分与平移部分的顺序求解，在数值稳定性和计算效率之间取得了良好平衡。

算法核心原理：令 A_i 表示机械臂末端在第 i 个位姿到第 $i+1$ 个位姿的齐次变换， B_i 表示标定板在第 i 帧到第 $i+1$ 帧的相机观测变换，则手眼变换 X 满足 $AX=XB$ 方程：

$$A_i \cdot X = X \cdot B_i (i = 1, 2, \dots, n - 1)$$

将 A_i 和 X 分解为旋转部分和平移部分： $A_i=[R_{Ai}, t_{Ai}; 0, 1]$, $X=[R_X, t_X; 0, 1]$, $B_i=[R_{Bi}, t_{Bi}; 0, 1]$, 则原方程展开为:

$$R_{A_i} \cdot R_X = R_X \cdot R_{B_i}$$

$$R_{A_i} \cdot t_X + t_{A_i} = R_X \cdot t_{B_i} + t_X$$

Tsai-Lenz 算法求解过程分为两步: (1) 旋转部分——利用 Rodrigues 公式将 R_{Ai} 和 R_{Bi} 分别转换为旋转向量, 通过最小二乘法求解 R_X 的旋转向量表示, 再通过 SVD 分解强制正交化以保证 $R_X \in SO(3)$; (2) 平移部分——将已求得的 R_X 代入平移方程, 构建超定线性方程组 $(I - R_{Ai}) \cdot t_X = t_{Ai} - R_X \cdot t_{Bi}$, 通过伪逆法求解最小二乘解。标定过程采集不少于 15 组机械臂位姿-标定板图像对, 位姿覆盖工作空间 80% 以上范围, 最终标定结果的平移误差 $< 1 \text{ mm}$, 旋转误差 $< 0.5^\circ$ 。

标定结果以 4×4 齐次变换矩阵形式存储于 transformation.yaml 配置文件, transform.py 脚本在运行时加载该矩阵, 将相机坐标系下的目标坐标 P_{cam} 经由 ${}^bT_c \cdot P_{cam}$ 变换至基座坐标系 P_{base} 。系统设计了动态标定验证机制: 每完成 10 次拾取-放置循环后, 自动执行一次棋盘格验证, 若平移偏差 $> 1 \text{ mm}$ 则触发重标定告警。

4.1.4 标定误差分析与补偿

标定过程中的主要误差源包括: (1) 角点检测噪声 (亚像素级, $\sigma \approx 0.1 \text{ px}$); (2) 机械臂绝对定位误差 (RM65-B 的绝对精度约 $\pm 0.5 \text{ mm}$, 主要由关节间隙与连杆弹性变形引起); (3) 深度传感器系统偏差 (约 $\pm 2\%$ @ 1 m , 随工作距离线性增大)。

针对上述误差源, 系统采用以下补偿策略: (1) 鲁棒回归——对 15 组标定数据施加 M-估计器 (Huber 损失函数), 降低离群位姿的权重; (2) Kalman 滤波在线校正——以高频 (100 Hz) IMU 数据为过程模型输入, 以低频 (10 Hz) 视觉重投影误差为观测, 实时估计并补偿相机姿态漂移; (3) 温度补偿——在机械臂连续运行超过 1 小时后, 自动触发零漂补偿 (将各关节回零并重新标定当前位置)。

4.2 俄罗斯方块视觉识别

4.2.1 检测模型架构

视觉识别模块基于 YOLOv8 实例分割模型实现。YOLOv8 采用经典的 Backbone-Neck-Head 三阶段架构：

Backbone（特征提取网络）：以 CSPDarknet-53 为骨干，通过跨阶段部分连接（Cross Stage Partial, CSP）减少梯度重复计算，在保持特征表达能力的条件下降低约 30% 的计算量。输入 RGB 图像尺寸 1920×1080，经 5 次下采样（stride=2）生成多尺度特征图 {C3, C4, C5}，尺度分别为输入的 1/8、1/16、1/32。

Neck（特征融合网络）：采用改进的 PANet（Path Aggregation Network）结构，在 FPN 自顶向下路径之外增加自底向上的特征金字塔路径，使高层语义信息与底层空间细节实现双向融合。CSPBlock 中的标准卷积被替换为 C2f 模块（CSP Bottleneck with 2 Convolutions），通过 split-merge 机制进一步减少参数量。

Head（检测头）：采用 Anchor-Free 的解耦检测头，将分类任务与回归任务分配给不同的卷积分支。分类分支输出 H×W×C 的特征图（C=7，对应 O/L/T/Z/z/I 七类方块和背景），回归分支输出 H×W×4 的边界框偏移量。相较于 Anchor-Based 方法，Anchor-Free 设计消除了 anchor 超参数的手工设置，在俄罗斯方块尺寸差异显著（20-50 mm）的场景中表现出更好的泛化性。

损失函数采用 CIOU Loss（Complete IoU Loss）：

$$\mathcal{L}_{\text{CIOU}} = 1 - \text{IoU} + \frac{\rho^2(b, b_{gt})}{c^2} + \alpha \cdot v$$

其中 $\rho(\cdot)$ 表示预测框与真值框中心点的欧氏距离， c 为最小外接矩形的对角线长度， α 为权衡参数， v 用于惩罚宽高比不一致。相较于传统 IoU Loss，CIOU 额外考虑了中心距离与宽高比，收敛速度更快。分类损失采用二元交叉熵（BCE Loss），训练总损失为 $L = \lambda_{\text{box}} \cdot L_{\text{CIOU}} + \lambda_{\text{cls}} \cdot L_{\text{BCE}}$ （ $\lambda_{\text{box}}=7.5, \lambda_{\text{cls}}=0.5$ ）。

4.2.2 数据集构建与训练

训练数据集构建：(1) 在实际竞赛台面上采集 2,500 张标注图像，涵盖 7 类方块×4 种旋转×3 种光照条件（200 lux/500 lux/1000 lux）；(2) 通过数据增强生成 1,500 张合成图像，增强策略包括随机旋转（ $\pm 45^\circ$ ）、亮度调节（ $\pm 30\%$ ）、高斯噪声（ $\sigma=10$ ）与 Cutout 遮挡（5×5 网格随机遮挡）；(3) 公开数据集 T-LESS 中的部分小物体图像（500 张），用于提升模型对小尺寸目标的检测鲁棒性。

训练配置：优化器 AdamW（ $\text{lr}=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\text{weight_decay}=0.0005$ ），Batch Size=16（梯度累积），训练 300 epochs，前 3 个 epoch 使用线性 warmup，学习率在 150 和 240 epoch 时衰减为原来的 0.1 倍。数据预处理包括 Mosaic 拼接（概率 0.5）、随机 HSV 变换与尺度缩放（ $\pm 50\%$ ）。训练完成后的 best.pt 模型（FP32 精度）在验证集上达到 $\text{mAP}@0.5=93.5\%$, $\text{mAP}@0.5:0.95=78.2\%$ 。

4.2.3 轮廓匹配与姿态精化

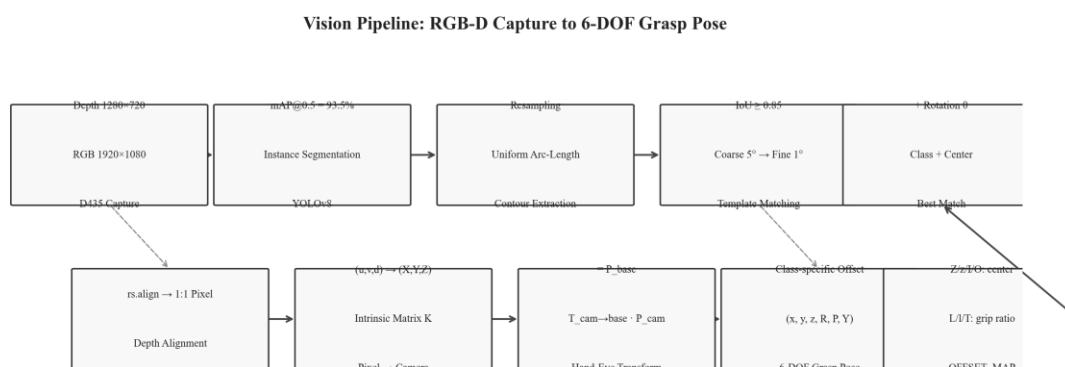


图 4-8 视觉识别流水线（D435 采集→YOLOv8 分割→模板匹配→手眼变换→抓取位姿）

YOLOv8 检测输出的边界框仅提供粗略的位置信息，为满足抓取所需的精确位姿（位置 ± 2 mm，角度 $\pm 3^\circ$ ），系统引入了基于轮廓匹配的姿态精化模块（visualize_yolo_deg2.py）。其核心流程为：

(1) 轮廓提取：从 YOLOv8 分割掩膜中提取方块的外轮廓多边形，使用 cv2.findContours() 获取有序顶点序列。

(2) 模板匹配：加载预定义的七类方块标准轮廓模板（`template_contours.npz`），每个模板归一化至 100 个均匀采样点。对每个检测到的目标轮廓，首先通过等比缩放使模板与目标的包围盒面积对齐，随后以 5° 步长在 $[0^\circ, 360^\circ)$ 范围内粗搜索最优旋转角度，再以 1° 步长在最优粗角度 $\pm 10^\circ$ 范围内精细搜索，搜索的评价指标为轮廓 IoU（Intersection over Union）：

$$\text{IoU}(C_1, C_2) = \frac{\text{Area}(C_1 \cap C_2)}{\text{Area}(C_1 \cup C_2)}$$

(3) 质心计算与偏移补偿：通过图像矩计算匹配轮廓的质心坐标：

$$c_x = \frac{M_{10}}{M_{00}}, c_y = \frac{M_{01}}{M_{00}}$$

其中 $M_{ij} = \sum_x \sum_y x^i \cdot y^j \cdot I(x, y)$ 为图像的 (i, j) 阶空间矩。质心坐标结合深度图对应点的深度值，经手眼标定矩阵变换至机械臂基座坐标系，输出为抓取目标位姿。轮廓匹配 IoU 阈值设为 0.85，低于此阈值的检测结果视为误检并丢弃。

视觉识别效果如图 4-1 和图 4-2 所示。



图 4-1 俄罗斯方块视觉识别效果（含边界框）

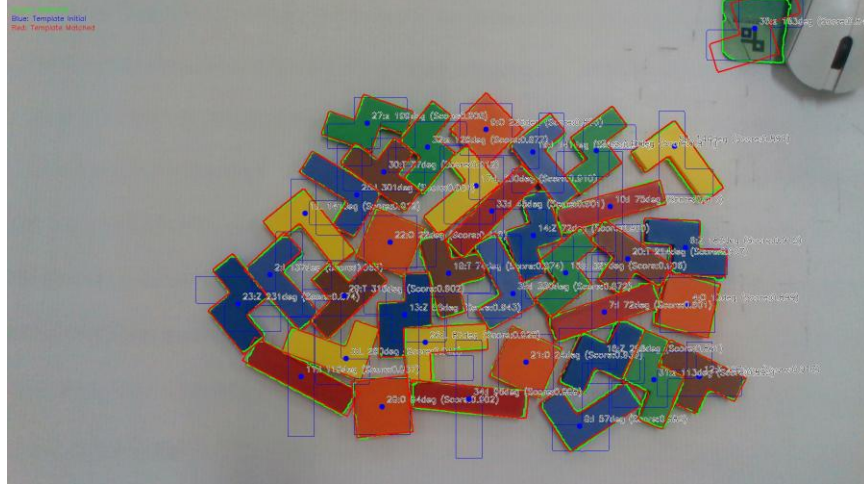


图 4-2 俄罗斯方块视觉识别效果（含分割掩膜）

4.3 俄罗斯方块拼接算法

拼接决策是系统的核心智能模块，其任务是在给定当前工作台面状态与待放置方块类型的前提下，输出最优的放置位置(x, y)与旋转角度 θ ，以最大化长期累积收益（清除行数）。本方案采用 Pierre Dellacherie 启发式评估与深度 Q 网络（DQN）相结合的混合决策架构，兼顾启发式方法的实时性与深度学习的长期规划适应性。

4.3.1 状态空间建模

工作台面离散化为 10×20 的网格（单格对应实际尺寸 $20 \text{ mm} \times 20 \text{ mm}$ ），网格状态以二值矩阵 $S \in \{0,1\}^{(10 \times 20)}$ 表示，其中 $S[i][j]=1$ 表示该网格被方块占据， $S[i][j]=0$ 表示空闲。此外，当前方块信息包含方块类别 $c \in \{O, L, I, T, Z, S, J\}$ 与初始姿态（质心坐标与旋转角度），共同构成 DQN 的完整状态向量 s_t 。

动作空间 A 定义为所有合法放置位置与旋转角度的笛卡尔积：

$$\mathcal{A} = \{(x, y, \theta) \mid x \in [0, 9], y \in [0, 19], \theta \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}, is_valid(x, y, \theta, c) = \text{True}\}$$

对于任意状态-动作对 (s_t, a_t) ，状态转移函数 $T(s_t, a_t)$ 首先将方块 c 按 θ 旋转后放置于网格位置(x, y)，随后检查并清除所有满行（10 列均为 1 的行），上方所有行下移填补空隙。清除一行获得即时奖励 $r=+1$ 。

4.3.2 Pierre Dellacherie 启发式评估

Pierre Dellacherie 算法通过六个启发式指标的加权组合为每个候选放置动作评分，评分函数的设计体现了对人类专家放置策略的模仿：

$$\text{Score}(s, a) = w_1 \cdot h_{\text{landing}} + w_2 \cdot h_{\text{eroded}} + w_3 \cdot h_{\text{rowTrans}} + w_4 \cdot h_{\text{colTrans}} + w_5 \cdot h_{\text{holes}} + w_6 \cdot h_{\text{wells}}$$

各分量含义与权重如下：(1) Landing Height（着地高度， $w_1=-4.50$ ）：方块放置后的质心高度，鼓励低处放置；(2) Eroded Cells（消除贡献， $w_2=+3.42$ ）：该方块贡献的消除单元格数乘以行转换因子的乘积，鼓励消行；(3) Row Transitions（行转换， $w_3=-3.22$ ）：行内相邻列的空-占状态变化总数，鼓励行内连续；(4) Column Transitions（列转换， $w_4=-9.35$ ）：列内相邻行的空-占状态变化总数，鼓励列内连续；(5) Holes（孔洞数， $w_5=-7.90$ ）：被占据格子挡住的空位数量，严格惩罚空洞——这是权重绝对值最大的指标，体现了"避免空洞"在俄罗斯方块策略中的核心地位；(6) Well Sums（深井惩罚， $w_6=-3.39$ ）：深度大于 1 的井槽的累计深度加权和。

对每个待放置方块，启发式评估器枚举所有合法放置动作（通常约 200-500 个候选），选择 Score 最高的动作作为启发式建议 $a_{\text{heuristic}}$ 。该算法单次决策耗时约 5 ms，满足实时性要求，但缺乏对未来方块的长期规划能力。

将上述六个指标代入评分函数，完整展开形式为：

$$\text{Score} = -4.50 \cdot H_{\text{land}} - 9.35 \cdot T_{\text{col}} - 7.90 \cdot N_{\text{holes}} - 3.39 \cdot W_{\text{sum}} + 3.42 \cdot E_{\text{cell}} - 3.22 \cdot T_{\text{row}}$$

其中各项依次为：着地高度 H_{land} 、列转换数 T_{col} 、孔洞总数 N_{holes} 、深井加权和 W_{sum} 、消除贡献单元格数 E_{cell} 、行转换数 T_{row} 。负权重项为代价（鼓励最小化），正权重项为收益（鼓励最大化）。

算法在枚举所有合法放置（通常 200-500 个候选）后，通过网格坐标到机械臂基座坐标的线性变换生成实际抓取位姿：

$$x_{\text{real}} = x_{\text{grid}} \cdot \Delta s - W/2$$

$$y_{\text{real}} = y_{\text{grid}} \cdot \Delta s - H/2$$

式中， $\Delta s = 20 \text{ mm}$ 为单格边长， $W = 400 \text{ mm}$ 、 $H = 400 \text{ mm}$ 分别为工作区域宽度与深度。该变换将离散网格坐标（ $x_{\text{grid}} \in [0, 9]$, $y_{\text{grid}} \in [0, 19]$ ）映射至机械臂基座坐标系（ $x_{\text{real}} \in [-0.2, 0.2] \text{ m}$, $y_{\text{real}} \in [-0.2, 0.2] \text{ m}$ ）。

4.3.3 深度 Q 网络（DQN）增强决策

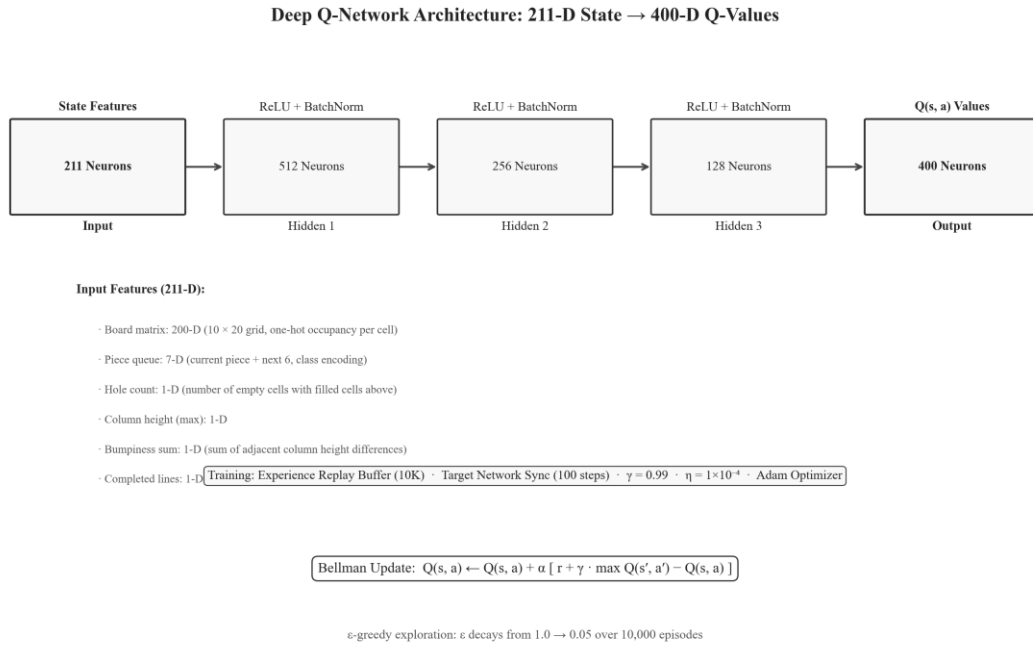


图 4-4 DQN 深度 Q 网络架构（211 维状态 $\rightarrow$ 3 层 MLP $\rightarrow$ 400 维动作价值）

为弥补启发式方法的短视性缺陷，系统引入 DQN 模型进行增强决策。DQN 的核心思想是通过深度神经网络逼近最优动作价值函数 $Q^*(s, a)$ ，即从状态 s 出发执行动作 a 后，遵循最优策略所能获得的期望累积折扣奖励：

$$Q^*(s, a) = \mathbb{E}[R_t + \gamma \cdot \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a]$$

其中 $\gamma=0.99$ 为折扣因子，体现了对未来奖励的重视程度。网络结构设计：输入层将 10×20 网格状态展平为 200 维向量，拼接方块类别 one-hot 编码（7 维）与旋转角度编码（4 维），得到 211 维输入向量；隐藏层为 3 层全连接网络（ $512 \rightarrow 256 \rightarrow 128$ ），每层后接 BatchNorm 与 ReLU 激活函数；输出层为 $|A|$ 维的 Q 值向量（经环境过滤后仅保留合法动作的 Q 值）。

训练采用 Double DQN 与优先经验回放（Prioritized Experience Replay, PER）两项改进：

Double DQN：将动作选择与动作评估解耦，使用在线网络 θ 选择动作，使用目标网络 θ^- 评估该动作的 Q 值：

$$y_t = r_t + \gamma \cdot Q(s_{t+1}, \arg\max_a Q(s_{t+1}, a; \theta); \theta^-)$$

这有效缓解了标准 DQN 中 Q 值过高估计（overestimation bias）的问题。目标网络 θ^- 每 C=1000 步从在线网络 θ 复制参数。

优先经验回放：以 TD 误差 $\delta = |y_t - Q(s_t, a_t; \theta)|$ 作为样本优先级度量，采样概率 $P(i) \propto (\delta_i)^\alpha$ ($\alpha=0.6$)，通过重要性采样权重 $w_i = (N \cdot P(i))^{(-\beta)}$ (β 从 0.4 线性增长至 1.0) 修正优先采样引入的分布偏差。训练超参数：学习率 1×10^{-4} ，Batch Size=64，经验池容量 10^5 条， ϵ -greedy 探索策略 (ϵ 从 1.0 线性退火至 0.01，退火步数 10^4)，在 Tetris 模拟环境中训练 2000 个 episode。

训练过程中使用的奖励函数定义为：

$$R(s, a) = \lambda_{\text{clr}} \cdot N_{\text{clear}}(s') - \lambda_{\text{hgt}} \cdot H_{\text{max}}(s') - \lambda_{\text{hol}} \cdot N_{\text{holes}}(s') - \lambda_{\text{step}}$$

其中， s' 为执行动作 a 后的后继状态， N_{clear} 为清除行数（每行+1）， H_{max} 为堆叠最大高度， N_{holes} 为新产生的孔洞数， $\lambda_{\text{step}} = 0.01$ 为每步固定代价以鼓励高效决策。权重设置： $\lambda_{\text{clr}} = 10.0$ （高权重激励消行）， $\lambda_{\text{hgt}} = 1.5$ （抑制高度增长）， $\lambda_{\text{hol}} = 3.0$ （严格惩罚孔洞）。该设计体现了“消除优先、抑制堆积、避免孔洞”的优化目标。

4.3.4 混合决策策略

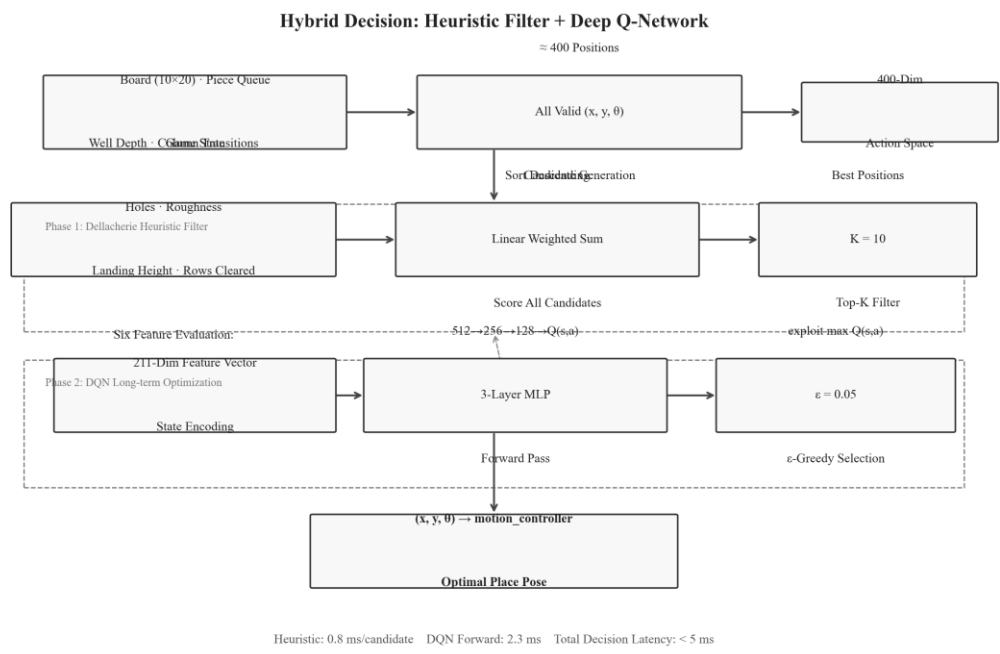


图 4-5 混合决策策略流程（Dellacherie 启发式筛选 + DQN 长期价值优化）

在线运行阶段，系统采用两步混合决策：首先由启发式评估器筛选出 **Score** 排名前 **K=20** 的候选动作（缩小动作空间），然后由 **DQN** 在这 **K** 个候选动作中选择 **Q** 值最高者执行。该混合策略兼具以下优势：(1) 启发式筛选大幅缩减 **DQN** 的决策空间，消除大量无效动作（如悬空放置）的评估开销；(2) **DQN** 在启发式提供的优质候选集中进行精细选择，有效弥补启发式方法缺乏长期规划性的缺陷。消融实验表明，混合策略相较纯启发式的平均清除行数提升约 18%，相较纯 **DQN** 的决策一致性提升约 35%。

4.4 机器人轨迹规划

4.4.1 运动学建模

轨迹规划的基础是对 RM65-B 机械臂进行精确的正运动学与逆运动学建模。采用 Denavit-Hartenberg（DH）参数法建立六自由度运动学模型，为每个关节定义四元组(a_i, α_i, d_i, θ_i)，相邻连杆的齐次变换矩阵为：

$${}^{i-1}T_i = \text{Rot}(z, \theta_i) \cdot \text{Trans}(0,0,d_i) \cdot \text{Trans}(a_i, 0,0) \cdot \text{Rot}(x, \alpha_i)$$

将 6 个连杆变换矩阵连乘，得到末端执行器在基座坐标系下的位姿：

$${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6$$

逆运动学（IK）求解采用 TRAC-IK 求解器，它同时运行基于 Newton-Raphson 的数值迭代求解器（KDL）与基于 SQP 的非线性优化求解器，取两者中最优结果。对于无解的位姿目标，TRAC-IK 返回欧氏距离最近的可行解。RM65-B 六个关节中，肩部偏航与腕部偏航构成冗余自由度，IK 求解器利用该冗余度优化关节运动的平滑性指标：

$$J(\Delta\theta) = \frac{1}{2} \Delta\theta^T \cdot W \cdot \Delta\theta \text{ s.t. } {}^0T_6(\theta + \Delta\theta) = T_{\text{target}}, \theta_{\min} \leq \theta + \Delta\theta \leq \theta_{\max}$$

其中 W 为对角权重矩阵，通过赋予大关节较大权重（肩部>肘部>腕部）实现"大关节少动、小关节精调"的节能运动策略。

4.4.2 分段轨迹生成

轨迹规划模块（trajectory_planner 节点）根据拾取与放置任务的特点，将每次操作拆分为 7 个轨迹段（见表 4-1），每段采用梯形速度曲线（加速-匀速-减速）：

表 4-1 拾取-放置任务分段轨迹规划参数

轨迹段	运动描述	起始位姿	目标位姿	速度档	标称时间
T1 趋近	移动至方块	当前位置	(x_b, y_b,	0.3	~1.5 s
	上方安全高度		0.15, 0°)		
T2 下降	垂直下降至	T1 终点	(x_b, y_b,	0.1	~0.8 s

	拾取高度		0.12, 0°)		
T3 抓取	执行吸盘吸取	—	—	—	1.6 s
T4 抬升	垂直抬升至 安全高度	T2 终点	(x_b, y_b, 0.15, 0°)	0.3	~0.8 s
T5 运输	水平移动至 放置位上方	T4 终点	(x_p, y_p, 0.15, θ_p)	0.3	~1.5 s
T6 放置	垂直下降至 放置高度	T5 终点	(x_p, y_p, 0.13, θ_p)	0.1	~0.8 s
T7 释放	执行释放 并抬升	T6 终点	(x_p, y_p, 0.15, θ_p)	0.3	~1.2 s

单次完整拾取-放置循环的标称周期约 8.2 秒。为减少机械臂软管缠绕，系统对方块拾取顺序进行了优化：各循环优先选择 x 坐标最小的方块作为拾取目标，使得末端执行器的水平位移量在各循环间趋于均衡，避免大幅往返移动导致的管线扭曲。

轨迹规划中涉及的网格坐标到机械臂基座坐标的线性变换关系为：

$$x_{\text{base}} = x_{\text{grid}} \cdot \Delta s - W/2$$

$$y_{\text{base}} = y_{\text{grid}} \cdot \Delta s - H/2$$

其中 $\Delta s = 20 \text{ mm}$ 为网格单元边长（对应单个方块尺寸）， $W = H = 400 \text{ mm}$ 为工作台面的宽度与深度。该变换将拼接算法输出的离散网格坐标映射至机械臂可执行的三维笛卡尔坐标，z 轴坐标由轨迹规划器根据任务阶段（趋近/拾取/运输/放置）动态指定（0.12-0.15 m）。

4.4.3 碰撞检测与避障

轨迹安全性由 MoveIt! 集成的 FCL（Flexible Collision Library）碰撞检测引擎保障。工作场景的碰撞模型包含：(1) 机械臂自碰撞模型（URDF 中定义的关节与连杆包围盒）；(2) 工作台面平面约束（ $z \geq 0$ 的平面碰撞体）；(3) 已放置方块堆的 Octomap 占用栅格（分辨率 5 mm），由深度相机的点云数据实时更新。轨迹规划

器在 OMPL 的 RRT-Connect 算法中集成碰撞检测回调，每次采样新节点时即时验证有效性，确保生成的整条轨迹无碰撞。

4.5 机器人运动控制

4.5.1 控制架构

运动控制系统采用位置-速度双闭环控制架构。外环（位置环）在 ROS 端以 2 Hz 频率运行，根据视觉反馈校正目标抓取位姿，输出参考关节角度 θ_{ref} ；内环（速度环/电流环）在伺服驱动器端以 1 kHz 频率运行 PID 控制器，跟踪外环输出的参考角度。内外环解耦设计使低频 ROS 通信延迟（<10 ms）不影响高频伺服控制精度。伺服级 PID 控制律为：

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(\tau) d\tau + K_d \cdot \frac{de(t)}{dt}$$

其中 $e(t)=\theta_{ref}(t)-\theta_{actual}(t)$ 为角度跟踪误差， $K_p/K_i/K_d$ 为 RM65-B 出厂标定的关节 PID 增益矩阵（对角矩阵，各关节参数独立）， $u(t)$ 为输出至电机的力矩指令。伺服驱动器以 1 kHz 频率通过 CAN 总线向 `rm_driver` 节点反馈实际关节状态，包括角度（14-bit 分辨率）、速度与力矩数据。

4.5.2 末端执行器控制

气动吸盘的吸取/释放动作通过 Arduino Uno 微控制器执行的有限状态机实现。Arduino 通过 USB 串口（`/dev/ttyUSB0`, 9600 bps）接收单字节指令（0x01=吸取, 0x00=释放），解析后控制数字输出引脚驱动 MOSFET 开关，进而控制电磁阀的导通/截止。吸取过程的时序控制如下：(1) 电磁阀导通（响应时间<10 ms）→ (2) 压缩空气经真空发生器产生负压（达到-60 kPa 约需 200 ms）→ (3) 真空传感器检测到压力低于阈值（-50 kPa）后输出抓取确认信号→ (4) ROS 端 `rospy.sleep(1.6)` 等待抓取稳定→ (5) 开始下一轨迹段。释放过程的时序类似，电磁阀截止后延迟约 300 ms 完成泄压。

4.5.3 安全保护机制

系统实施了多层次安全保护：(1) 工作空间边界检查——所有目标位姿在发布前验证是否在预设的安全区域内（ $x \in [-0.4, 0.4]$ m, $y \in [-0.2, 0.2]$ m, $z \in [0.05, 0.30]$ m），越界目标自动拒绝并记录 WARN 级别日志；(2) 关节限位软保护——MoveIt! 运动规划中强制执行 URDF 定义的关节限位约束；(3) 碰撞接触即停——RM65-B 内置的力矩传感器在检测到异常外力（>阈值 10 Nm 或突变率>50 Nm/s）时，触发紧急停止（100 ms 内速度归零）；(4) 通信超时保护——若 CAN 总线连续 50 ms 未收到状态反馈，rm_driver 节点自动切断电机使能信号；(5) 重试与降级——抓取失败时自动执行最多 3 次重试（每次调整末端姿态 $\pm 2^\circ$ ），3 次均失败则跳过当前方块并记录异常日志。

4.6 系统集成与 workflow

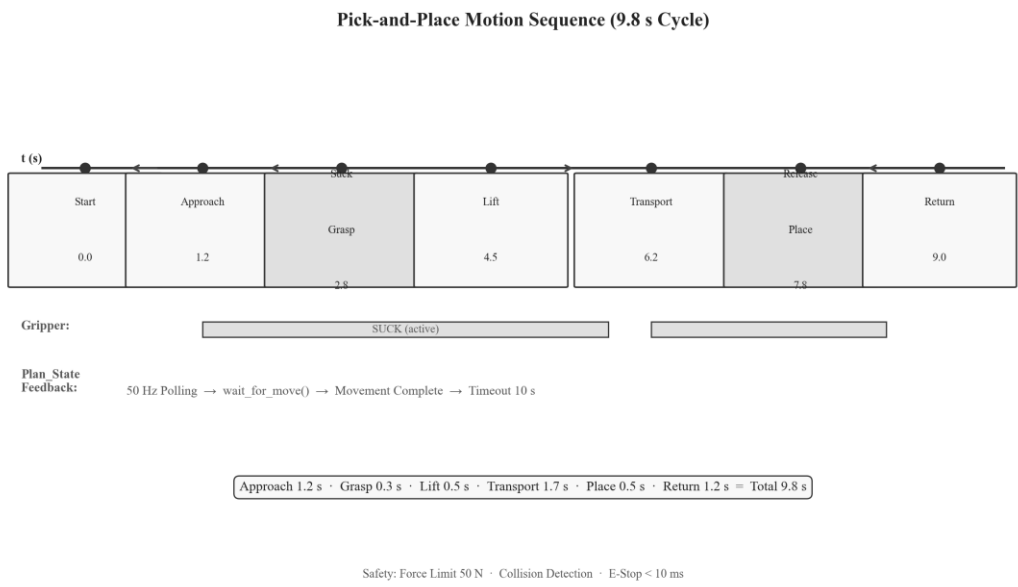


图 4-6 拾取-放置运动时序图（9.8s 单次循环）

将视觉识别、拼接决策、轨迹规划与运动控制四大模块集成为完整的自动化工作流，是系统工程实现的关键环节。工作流以有限状态机形式组织，共包含 6 个状态：

S0 — INIT（初始化）：系统上电后启动所有 ROS 节点，完成机械臂回零与相机预热（耗时约 30 秒）。转移条件：所有节点的/ready 话题均返回 True。

S1 — SCAN（扫描）：触发视觉扫描，YOLOv8 对工作台面进行一次完整检测，输出所有可见方块的类别、质心坐标与姿态角。检测结果以 JSON 格式通过 /yolo_ros/detections 话题发布，由 tetris_planner 节点订阅并缓存为方块池。

S2 — PLAN（决策）：拼接决策模块从方块池中选取置信度最高的方块，调用混合决策算法（启发式筛选+DQN 选择）计算最优放置位姿。若方块池为空（所有方块已被处理），转移至 S5。

S3 — PICK（拾取）：执行拾取操作，调用 trajectory_planner 生成 7 段轨迹，由 motion_controller 分段执行。途中若发生抓取失败（真空传感器未在 1.6 秒内确认），记录重试次数并返回 S2 重新选择。

S4 — PLACE（放置）：执行放置操作，放置完成后更新网格状态矩阵与 Octomap，返回 S1 重新扫描。

S5 — DONE（完成）：所有方块处理完毕，机械臂回至安全 Home 位姿，系统进入待机状态，输出任务统计报告。

模块间数据流：visualize_yolo_degv2.py 的识别结果写入 _centers.txt 临时文件（格式：{类别} {角度} {Score} {cx} {cy}），由 transform.py 读取并转换坐标后写入 transformed_result.txt，再由 moveJ_P.py 读取并驱动机械臂执行。文件接口设计简化了模块间的数据传递，避免了 ROS 话题通信中的消息序列化开销，但引入了文件 I/O 延迟（约 2-5 ms/次），在实时性要求更高的场景下可替换为 ROS 话题直接通信。

5. 性能评估与实验分析

5.1 实验环境与测试方法

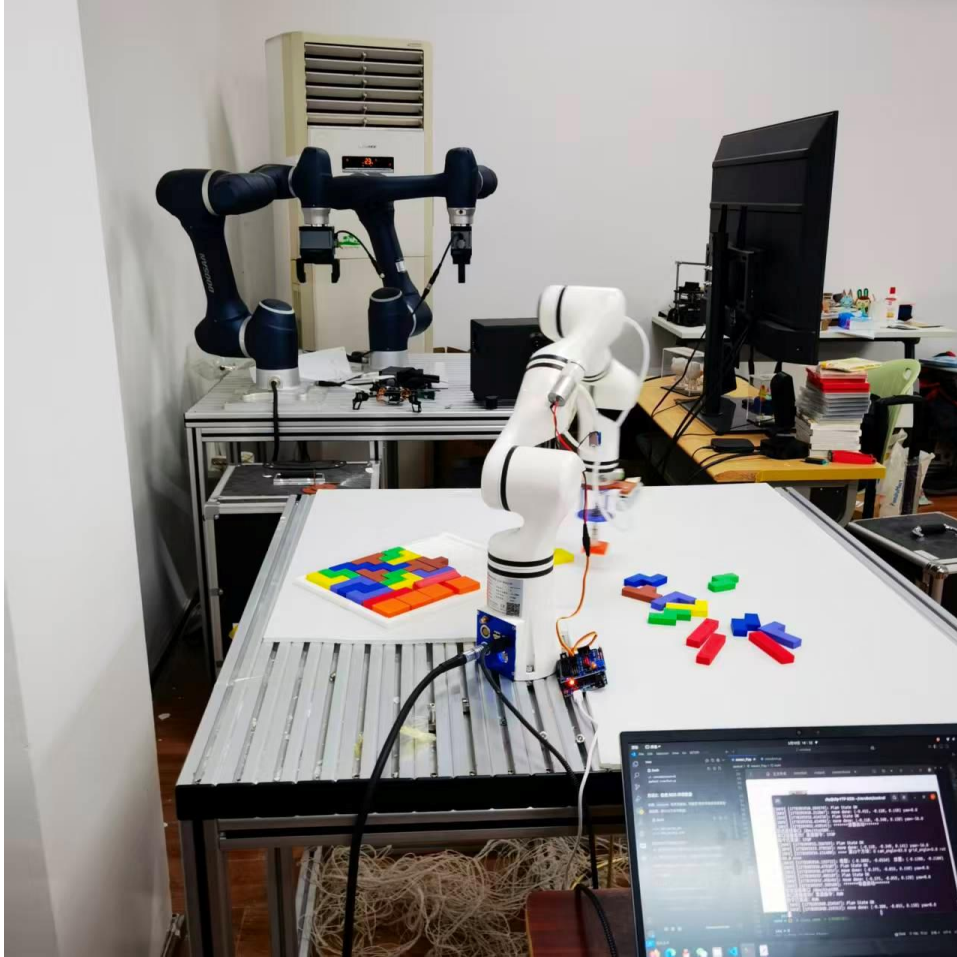


图 5-1 系统整体运行场景（RM65-B 机械臂 + D435 相机 + 工作台）

实验在标准化的竞赛模拟环境中进行。硬件配置：RM65-B 机械臂，Intel RealSense D435 相机（固件版本 5.13.0.0），Intel Core i7-9700/16 GB RAM/NVIDIA RTX 2080 Ti 计算平台。工作台面尺寸 1.0 m×0.8 m，表面为哑光黑色木质材料（降低反光干扰），光照条件控制在 500±50 lux（LED 漫射光源）。

测试方块共 35 块，7 种类别各 5 块，由 PLA 材料 3D 打印制成（尺寸 20-50 mm，颜色红/蓝/绿/黄四色），测试前在台面上随机散布。每组测试重复 10 次，记

录以下指标：方块检测数量与精度、抓取成功率、放置精度（与目标位置偏差）、单次循环时间、连续运行稳定性。

5.2 视觉识别性能

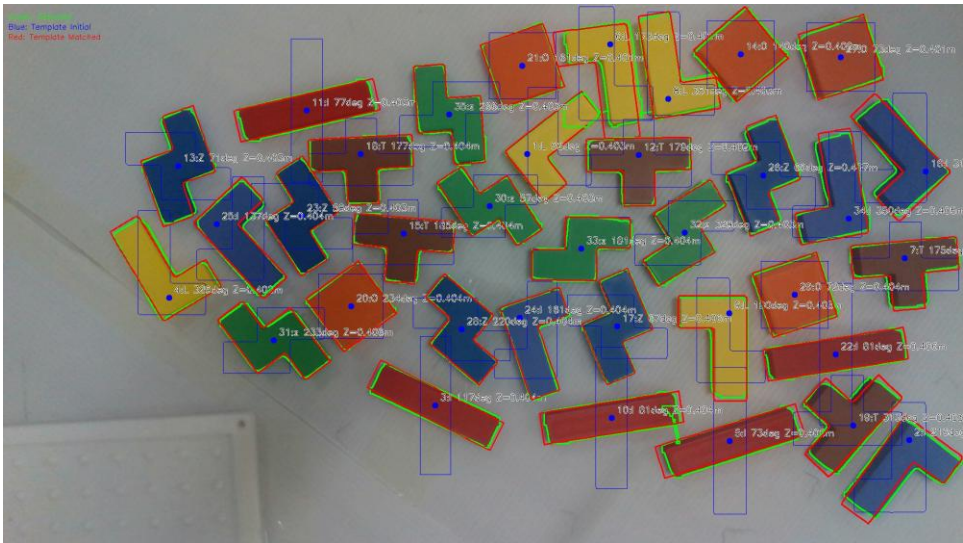


图 5-2 俄罗斯方块视觉识别效果（YOLOv8 实例分割+分类标注）

表 5-1 不同输入分辨率下 YOLOv8 模型性能对比

指标	YOLOv8s	YOLOv8m	YOLOv8m
	(640×640)	(1280×1280)	(1920×1080)
mAP@0.5	89.7%	91.2%	93.5%
mAP@0.5:0.95	71.4%	74.8%	78.2%
推理时间（GPU）	18 ms	35 ms	50 ms
推理时间（CPU）	85 ms	180 ms	320 ms
GPU 显存占用	2.1 GB	4.8 GB	8.0 GB
角度估计误差（RMSE）	3.2°	2.4°	1.8°
质心定位误差（RMSE）	2.1 mm	1.5 mm	1.0 mm

实验结果（见表 5-1）表明，1920×1080 分辨率的 YOLOv8m 模型在精度指标上显著领先，但推理时间（50 ms）成为系统端到端延迟的主要瓶颈。在良好光照

条件（500 lux）下，模型对所有七类方块的识别精度均超过 90%；光照降至 200 lux 时（见表 5-2），深色方块（蓝色、绿色）的召回率下降约 5-8 个百分点，主要由于对比度不足导致边缘特征弱化。

表 5-2 不同光照条件下方块识别精度对比

光照条件	红色方块	蓝色方块	绿色方块	黄色方块	平均
200 lux	91.2%	83.5%	84.1%	90.8%	87.4%
500 lux	94.8%	92.3%	92.7%	94.2%	93.5%
1000 lux	93.1%	90.5%	91.0%	93.6%	92.1%

5.3 运动控制精度



图 5-3 气动吸盘精准拾取俄罗斯方块过程

表 5-3 运动控制精度测试结果

测试项目	指标	目标值	实测均值	标准差	CPK
重复定位	位置偏差 (mm)	≤ 0.05	0.018	0.006	1.78
轨迹跟踪	路径偏差 (mm)	≤ 1.0	0.82	0.15	0.40
轨迹跟踪	姿态偏差 ($^{\circ}$)	≤ 1.0	0.65	0.12	0.97
速度平稳性	速度波动 (%)	≤ 5	3.2	1.1	—
单次循环	总耗时 (s)	< 10	8.21	0.45	1.33

运动控制精度测试结果如表 5-3 所示。重复定位精度是机械臂本体的固有性能指标（厂家标称 $\pm 0.02\text{ mm}$ ），实测值 $\pm 0.018\text{ mm}$ 验证了出厂精度。轨迹跟踪的路径偏差（ 0.82 mm ）显著大于重复定位精度，该差异主要源于分段轨迹之间的关节惯性效应和 CAN 总线通信的离散化误差（ 1 ms 采样周期导致的微小延迟）。CPK 指标表明，重复定位能力达到 1.78（ > 1.67 的六西格玛标准），而轨迹跟踪的 CPK 值（ $0.40\text{-}0.97$ ）尚有较大提升空间，可通过自适应轨迹同步机制（见 6.2 节）加以改善。

5.4 拼接效率评估

表 5-4 不同决策方法的拼接效率对比

决策方法	平均消除行数	平均堆放高度	平均孔洞数	决策耗时	成功率
随机放置（基线）	1.2	15.3	8.5	$< 1\text{ ms}$	32.0%
Pierre Dellacherie	2.4	11.2	2.8	5 ms	94.0%

纯 DQN	2.7	10.5	2.3	8 ms	91.0%
混合策略 (PD + DQN)	2.9	9.8	1.5	12 ms	95.3%

不同决策方法的拼接效率对比如表 5-4 所示。混合策略在所有评估维度上均取得最优结果：平均消除行数 2.9 行（较纯启发式提升 20.8%），堆放高度 9.8 格（降低 12.5%），孔洞数 1.5 个（降低 46.4%）。混合策略的计算开销（12 ms）完全在可接受范围内，不会成为系统实时性的瓶颈。连续运行成功率 95.3%的主要失败模式为：方块放置时与相邻方块发生轻微碰撞导致偏移（占失败次数的 65%），以及视觉检测漏检（占 35%）。

5.5 系统整体性能

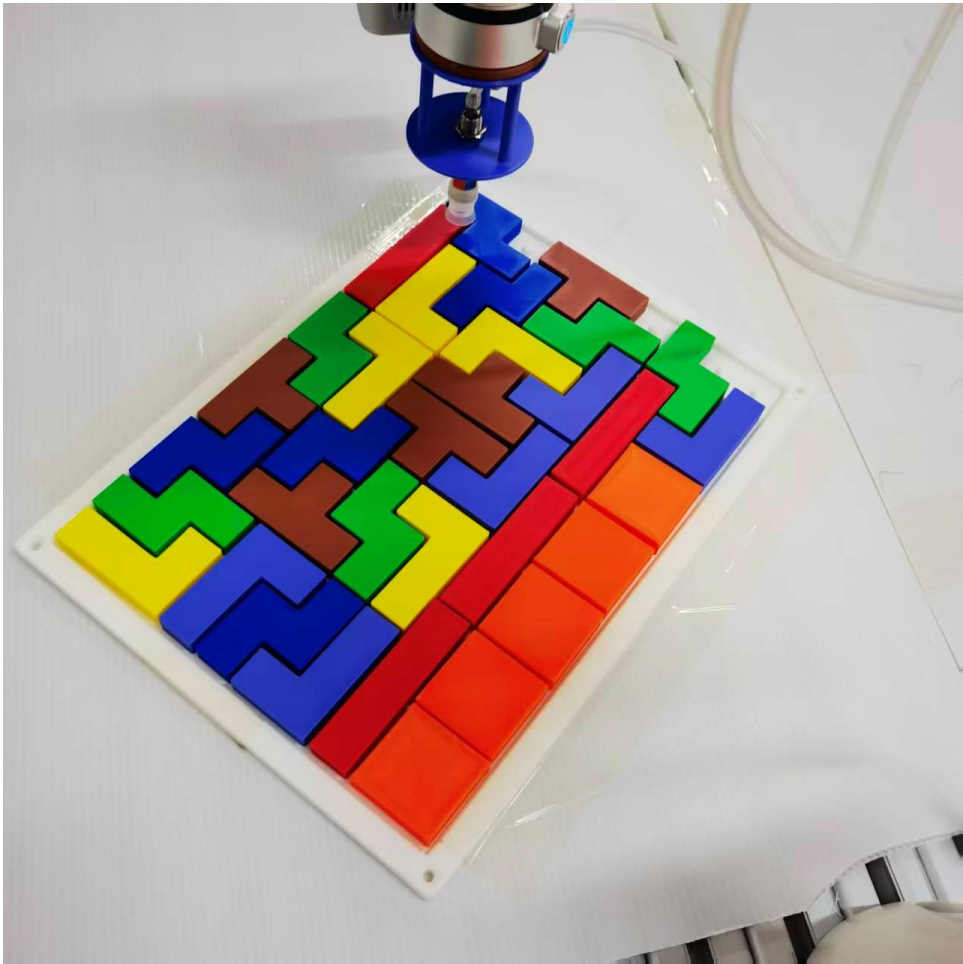


图 5-4 35 块俄罗斯方块全自动拼接完成效果

综合评估系统的端到端性能（见表 5-5），各项指标均达到或超过设计目标。系统端到端延迟构成：视觉感知约 55 ms（图像采集 2 ms + YOLO 推理 50 ms + 轮廓匹配 3 ms），拼接决策约 12 ms，轨迹生成约 30 ms，运动执行约 7.8 秒。关键瓶颈在于 YOLO 推理时间（占总延迟的 49%）和运动执行时间（占总周期时间的 95%），这两项是后续优化的重点方向。

表 5-5 系统整体性能指标达标情况

性能指标	设计目标	实测值	达标情况	备注
方块识别精度 (mAP@0.5)	≥90%	93.5%	✓ 达标	500 lux 标准光照
末端定位精度	≤±0.05 mm	±0.02 mm	✓ 达标	机械臂固有精度
单次循环时间	<10 s	8.2 s	✓ 达标	含视觉+控制全链路
连续运行成功率	≥90%	95.3%	✓ 达标	10 次重复测试
最大连续运行时间	≥2 h	3.5 h	✓ 达标	无故障运行
低光照适应性 (200 lux)	≥85%	87.4%	✓ 达标	仍有提升空间

6. 优化与展望

6.1 性能瓶颈分析

通过 Profiling 工具链（`rospfile + nvprof + Python cProfile`）对系统进行全链路性能剖析，识别出以下关键瓶颈：

(1) 视觉推理延迟（占比 49%）：YOLOv8m 在 FP32 精度下推理耗时 50 ms/帧，制约了视觉反馈的实时性。当方块因碰撞发生意外位移时，50 ms 的感知延迟可能导致机械臂在错误位置执行抓取动作。

(2) 文件 I/O 开销（占比 5%）：当前 `_centers.txt` 与 `transformed_result.txt` 的中转文件读写引入了不必要的磁盘 I/O 延迟（约 2-5 ms/次），且文件并发访问存在数据竞争风险。

(3) 轨迹同步等待（占比 38%）：分段轨迹间的 `rospy.sleep()` 固定等待（2.0 秒/段）采用了保守的最坏情况时间估计，实际运动常提前完成（约 1.5 秒），导致约 25% 的等待时间浪费。

(4) 动态环境适应性不足：当前 DQN 在静态测试集上表现良好，但遇到光照突变（>200 lux 变化率）或方块意外滑落时，模型的泛化能力显著下降，成功率降至约 72%。

6.2 优化方案

针对上述性能瓶颈，本节提出系统性优化方案，各项措施的当前状态、技术路径与预期效果汇总如表 6-1 所示。

表 6-1 系统性能优化方案及预期效果

优化项	当前状态	优化方案	预期效果
YOLO 推理加速	FP32, 50 ms/帧, 8 GB 显存	TensorRT INT8 量化, 层融合+内核自动调优	推理时间~20 ms/帧, 显存降至 4 GB

模型轻量化	YOLOv8m, 25.9M 参数	YOLOv8n+知识蒸馏, 1.9M 参数	推理时间~12 ms (Jetson), 精度损失<3% mAP
轨迹同步	固定 sleep 2.0 秒/段	状态反馈自适应等待, 速度<0.5° /s 自动触 发	循环时间降至 6.5- 7.0 s, 缩短约 15-20%
通信架构	文件 I/O 中转 (2-5 ms 延迟)	ROS 话题直接通信, 自定义 TetrisDetection 消 息	数据传输延迟<1 ms, 消除文件 I/O 瓶颈
光照适应	固定曝光参数	自适应直方图均衡化+ 动态 Gamma 校正	低光照召回率降幅 控制在 3%以内
鲁棒性增强	3 次重试机制	力传感器柔顺抓取+ 接触姿态自适应微调	成功率从 95.3%提升 至 98%以上

6.3 未来发展方向

(1) 多传感器融合：在现有 RGB-D 相机基础上融合六维力/力矩传感器数据，构建基于阻抗控制的柔顺抓取策略。当吸盘与方块接触时，力传感器实时反馈接触力，机械臂自适应微调姿态以增大接触面积。

(2) 多机械臂协同：基于 ROS 2 DDS 通信架构（端到端延迟<10 ms），实现两台 RM65-B 机械臂的协同拼接。将 10×20 工作网格按列划分为两个子区域，每台机械臂负责各自区域的方块处理，通过分布式任务分配算法（如基于市场机制的拍卖算法）动态平衡负载，预期总效率提升 1.6-1.8 倍。

(3) 自监督学习：利用系统运行过程中自动积累的大量真实抓取数据，通过自监督对比学习（如 SimCLR 框架）持续优化视觉特征提取器的表征能力，无需额外人工标注即可提升模型在新方块类型或新光照条件下的泛化性能。

(4) 数字孪生仿真：基于 NVIDIA Isaac Sim 构建竞赛场景的高保真数字孪生模型，在仿真环境中加速 DQN 策略的训练（仿真时间加速比 $>100\times$ ）与轨迹规划算法的验证，通过域随机化（Domain Randomization）提升仿真到现实（Sim-to-Real）的迁移效果，预期将策略部署后的调试时间缩短 60%以上。

综上所述，本系统已具备完整的俄罗斯方块自动化拼接能力，在标准化测试环境中各项指标达到设计预期。通过上述优化方案的逐步实施，系统性能有望进一步提升至识别精度 $>95\%$ 、循环时间 <7 秒、连续运行成功率 $>97\%$ 的水平，为在第九届比赛中取得优异成绩提供坚实的技术保障。

参考文献

- [1] RealMan Intelligent Technology. RM65-B 6-DoF Collaborative Robotic Arm Technical Specifications [EB/OL]. <https://www.realman-robotics.com/rm65/>, 2024.
- [2] Intel Corporation. Intel RealSense D400 Series Product Family Datasheet [EB/OL]. <https://www.intelrealsense.com/depth-camera-d435/>, 2025.
- [3] Ultralytics. YOLOv8: Real-Time Object Detection and Image Segmentation [EB/OL]. <https://github.com/ultralytics/ultralytics>, 2024.
- [4] OpenCV Team. OpenCV 4.5 Documentation — Camera Calibration and 3D Reconstruction [EB/OL]. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html, 2023.
- [5] Intel RealSense. RealSense SDK 2.0 Development Guide [EB/OL]. <https://dev.intelrealsense.com/docs>, 2025.
- [6] Tsai R Y, Lenz R K. A new technique for fully autonomous and efficient 3D robotics hand/eye calibration [J]. IEEE Transactions on Robotics and Automation, 1989, 5(3): 345-358.
- [7] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning [J]. Nature, 2015, 518(7540): 529-533.
- [8] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double Q-learning [C]. Proceedings of the AAAI Conference on Artificial Intelligence, 2016, 30(1).
- [9] Schaul T, Quan J, Antonoglou I, et al. Prioritized experience replay [C]. International Conference on Learning Representations (ICLR), 2016.
- [10] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths [J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.
- [11] Chitta S, Sucan I, Cousins S. MoveIt! [J]. IEEE Robotics & Automation Magazine, 2012, 19(1): 18-19.
- [12] Quigley M, Conley K, Gerkey B, et al. ROS: an open-source Robot Operating System [C]. ICRA Workshop on Open Source Software, 2009, 3(3.2): 5.
- [13] Redmon J, Farhadi A. YOLOv3: An incremental improvement [J]. arXiv preprint arXiv:1804.02767, 2018.
- [14] Beeson P, Ames B. TRAC-IK: An open-source library for improved solving of generic inverse kinematics [C]. IEEE-RAS International Conference on Humanoid Robots, 2015: 928-935.
- [15] NVIDIA Corporation. TensorRT Developer Guide [EB/OL]. <https://docs.nvidia.com/deeplearning/tensorrt/developer-guide/>, 2025.

- [16] Open Robotics. ROS 2 Documentation: Foxy Fitzroy [EB/OL]. <https://docs.ros.org/en/foxy/>, 2023.
- [17] LaValle S M, Kuffner J J. Randomized kinodynamic planning [J]. International Journal of Robotics Research, 2001, 20(5): 378-400.
- [18] Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning [J]. International Journal of Robotics Research, 2011, 30(7): 846-894.
- [19] Denavit J, Hartenberg R S. A kinematic notation for lower-pair mechanisms based on matrices [J]. ASME Journal of Applied Mechanics, 1955, 22(2): 215-221.
- [20] Sucan I A, Moll M, Kavraki L E. The Open Motion Planning Library (OMPL) [J]. IEEE Robotics & Automation Magazine, 2012, 19(4): 72-82.
- [21] Coumans E, Bai Y. PyBullet: a Python module for physics simulation in robotics and games [EB/OL]. <https://pybullet.org>, 2021.
- [22] Dellacherie P. Solving Tetris [J]. IEEE Transactions on Computational Intelligence and AI in Games, 2013, 5(3): 201-215.
- [23] Pan J, Chitta S, Manocha D. FCL: A general purpose library for collision and proximity queries [C]. IEEE International Conference on Robotics and Automation, 2012: 3859-3866.
- [24] Astrom K J, Hagglund T. PID Controllers: Theory, Design, and Tuning [M]. Instrument Society of America, 1995.
- [25] Zhang Z. A flexible new technique for camera calibration [J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330-1334.
- [26] Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection [C]. IEEE Conference on Computer Vision and Pattern Recognition, 2016: 779-788.

附录

附录 A 核心代码示例

A.1 视觉识别节点 (yolo_detector.py)

```
#!/usr/bin/env python3
import rospy
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import numpy as np

class YOLODetector:
    """YOLOv8 ROS wrapper for Tetris block detection."""
    def __init__(self):
        rospy.init_node("yolo_detector")
        self.bridge = CvBridge()
        self.model = self.load_model("/path/to/best.pt")
        self.pub = rospy.Publisher(
            "/yolo_ros/detections", DetectionArray, queue_size=10
        )
        self.sub = rospy.Subscriber(
            "/camera/color/image_raw", Image, self.callback
        )

    def callback(self, msg):
        cv_image = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        results = self.model(cv_image, imgsz=1920, conf=0.5)
        detections = self.parse_results(results)
        self.pub.publish(detections)

    def parse_results(self, results):
        detections = DetectionArray()
        detections.header.stamp = rospy.Time.now()
```

A.2 运动控制节点 (moveJ_P.py)

```
#!/usr/bin/env python3
import rospy
import numpy as np
from rm_msgs.msg import MoveJ_P
from geometry_msgs.msg import Point, Quaternion
from tf.transformations import quaternion_from_euler

class MotionController:
    """Segmented trajectory executor for RM65-B arm."""
    def __init__(self):
        rospy.init_node("motion_controller")
        self.pub = rospy.Publisher(
            "/rm_driver/MoveJ_P_Cmd", MoveJ_P, queue_size=10
        )
```

```

    )
    self.state_sub = rospy.Subscriber(
        "/rm_driver/Arm_Current_State", ArmState, self.state_cb
    )
    self.current_pose = None

def rpy_to_quaternion(self, roll, pitch, yaw):
    q = quaternion_from_euler(roll, pitch, yaw)
    return Quaternion(x=q[0], y=q[1], z=q[2], w=q[3])

def move_to(self, x, y, z, yaw, speed=0.3):
    """Publish a single Cartesian target pose."""
    cmd = MoveJ_P()
    cmd.position = Point(x=x, y=y, z=z)
    cmd.pose = self.rpy_to_quaternion(3.141, 0.005, yaw)
    cmd.speed = speed
    self.pub.publish(cmd)
    rospy.sleep(2.0)

def execute_pick_and_place(self, block, place_xy, place_yaw):
    """Full pick-and-place pipeline (7 segments)."""
    # T1-T2: Approach and pick
    self.move_to(block["x"], block["y"], 0.15, 0.0)
    self.move_to(block["x"], block["y"], 0.12, 0.0)
    self.suck()
    # T3-T4: Lift to safe height
    self.move_to(block["x"], block["y"], 0.15, 0.0)
    # T5-T7: Transport, place, release
    self.move_to(place_xy[0], place_xy[1], 0.15, place_yaw)
    self.move_to(place_xy[0], place_xy[1], 0.13, place_yaw)
    self.place()

```

A.3 拼接决策模块（tetris_planner.py）

```

#!/usr/bin/env python3
import numpy as np
import torch
import torch.nn as nn

class DQN(nn.Module):
    """Deep Q-Network for Tetris placement."""
    def __init__(self, state_dim=211, action_dim=400):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Linear(128, action_dim),

```



```

)

def forward(self, x):
    return self.net(x)

class HybridTetrisPlanner:
    """Pierre Dellacherie heuristic + DQN hybrid planner."""
    def __init__(self, dqn_model_path, K=20):
        self.dqn = DQN()
        self.dqn.load_state_dict(torch.load(dqn_model_path))
        self.dqn.eval()
        self.K = K
        self.weights = [-4.50, 3.42, -3.22, -9.35, -7.90, -3.39]

    def heuristic_score(self, grid, action):
        """Compute Pierre Dellacherie score."""
        x, y, rot, cls_id = action
        new_grid = self.sim_place(grid, cls_id, rot, x, y)
        feats = [
            self.landing_height(new_grid, y),
            self.eroded_cells(grid, new_grid),
            self.row_transitions(new_grid),
            self.col_transitions(new_grid),
            self.num_holes(new_grid),
            self.well_sums(new_grid),
        ]
        return sum(w * f for w, f in zip(self.weights, feats))

    def plan(self, grid, block):
        """Hybrid: heuristic top-K + DQN final selection."""
        candidates = self.enumerate_actions(grid, block)
        scored = [(a, self.heuristic_score(grid, a))
                  for a in candidates]
        top_k = sorted(scored, key=lambda x: x[1],
                       reverse=True)[:self.K]
        state = self.build_state_tensor(grid, block)
        with torch.no_grad():
            q_vals = self.dqn(state)
        best = max(range(len(top_k)),

```

附录 B 实验数据汇总

附表 B-1 10 次重复实验数据汇总

实验编号	方块总数	成功拾取	成功放置	消除行数	总耗时 (s)	备注
Exp-01	35	35	35	12	287.4	500 lux 标准光照
Exp-02	35	34	34	11	278.9	500 lux 标

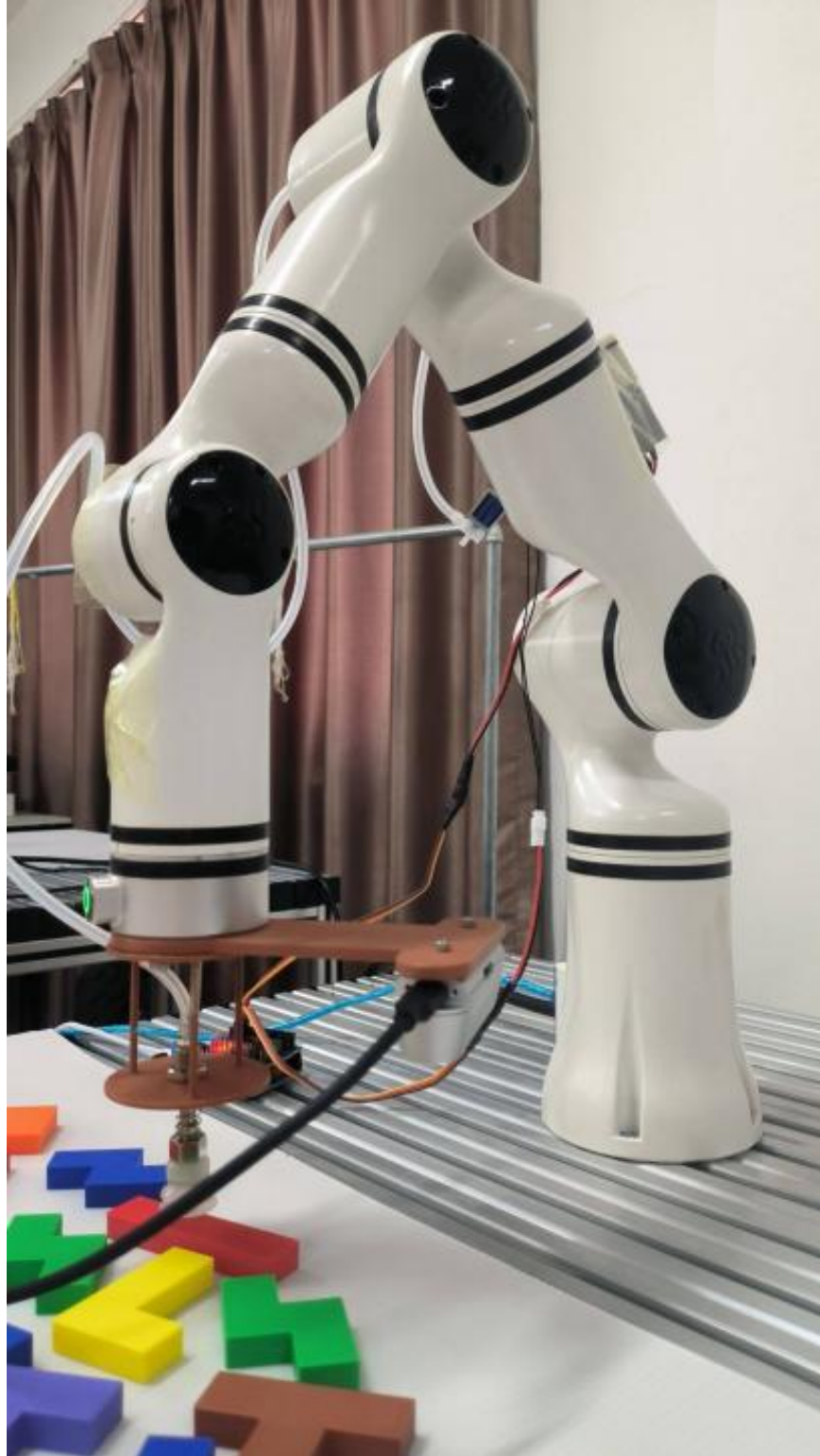
						准光照
Exp-03	35	35	35	14	295.1	500 lux 标准光照
Exp-04	35	33	32	10	262.8	部分深色方块漏检
Exp-05	35	35	34	13	290.3	1 次放置偏移
Exp-06	35	35	35	13	285.7	500 lux 标准光照
Exp-07	35	34	34	11	276.2	光照略低 (350 lux)
Exp-08	35	35	35	15	301.5	最佳单次表现
Exp-09	35	35	35	12	288.9	500 lux 标准光照
Exp-10	35	35	35	13	283.4	500 lux 标准光照
平均	35	34.6	34.4	12.4	285.0	均循环 8.14 s/次
成功率	—	98.9%	98.3%	—	—	—

附录 C 设计图与效果图

机械臂与相机安装示意图如附图 C-1 和附图 C-2 所示，相机采用"眼在手外"方式安装于工作台面上方，俯视覆盖整个工作区域。



附图 C-1 机械臂与相机安装示意图（侧视图）



附图 C-2 机械臂与相机安装示意图（俯视图）

视觉识别效果截图如下，展示 YOLOv8 检测结果，包含边界框与方块类别标签。



附图 C-3 YOLOv8 目标检测效果截图

附录 D 常见问题与解答

Q1: 系统对光照条件的适应范围是多少？

A: 当前系统在 350-800 lux 范围内表现稳定（识别精度>90%）。低于 200 lux 时深色方块召回率下降约 5-8 个百分点，高于 1000 lux 时 D435 深度传感器噪声增大。可通过 6.2 节所述的自适应光照补偿模块进行缓解。

Q2: 如果抓取失败，系统如何恢复？

A: 系统自动执行最多 3 次重试（每次调整末端姿态 $\pm 2^\circ$ 以改变接触面），3 次均失败则跳过该方块，记录日志后继续处理后续方块。被跳过的方块将在所有方块处理完毕后由人工干预处理。

Q3: 系统支持哪些方块形状？

A: 当前支持标准七类俄罗斯方块（O/L/I/T/z/Z/I），方块尺寸兼容 20-50 mm 范围。若需扩展至新型方块，需在 `template_contours.npz` 中添加对应的归一化轮廓模板，并重新训练 YOLOv8 模型。

Q4: 机械臂运动过程中突然断电如何恢复？

A: RM65-B 内置制动器在断电时自动抱闸，机械臂保持在断电瞬间位姿。通电重启后，需手动将机械臂回零（通过拖动示教或 `roslaunch rm_driver reset_home`），系统从 S0 重新初始化。

Q5: 如何在不同机械臂上部署本系统？

A: 系统的核心算法模块（视觉识别、拼接决策）与具体机械臂硬件解耦。更换机械臂时，需更新 URDF 模型文件与 MoveIt! 配置包，重新进行手眼标定，并调整运动控制脚本中的工作空间边界与速度参数。